

# ASTRA MOVE

Sistema de Controle e Monitoramento Operacional da Base Lunar

Turma: 1ECATemas: Base e Residência & Mobilidade

INTEGRANTES DO GRUPO:

Anderson Luiz Sanches Junior — RM: 571834

Vitor Andrade de Oliveira — RM: 570907

João Pedro Moraes Basques — RM: 571710

Gabriel de Oliveira — RM: 569455

Yagu naoto Iwanaga RM: 570825

## 1. VISÃO GERAL DOS TEMAS DO PROJETO

O projeto da Astra Move aborda o controle e monitoramento operacional completo de uma infraestrutura espacial, focado nos dois temas principais escolhidos pela equipe:

Tema 1: Base e Residência

Monitoramento do suporte à vida nos habitats lunares (níveis de  $O_2$  e  $CO_2$ ), ocupação das acomodações, controle de capacidade por módulo e gerenciamento de situações ambientais críticas.

Tema 2: Mobilidade

Gerenciamento de frota de rovers operacionais, verificação de autonomia e percentual de bateria, análise de terrenos perigosos/seguros e simulação de missões de transporte de carga e rotas.

## 2. RESUMO DA ARQUITETURA DOS PROGRAMAS

Programa 1: Diagnóstico e Alertas (Autônomo)

Varre automaticamente os módulos de residência e os veículos da base. Se o nível de  $CO_2$  exceder 1000 ppm ou o  $O_2$  cair abaixo de 20.5%, aciona alerta crítico para evacuação ou realocação para módulos livres. Também avalia rovers aptos e o mapa de terrenos.

Programa 2: Simulador de Missões (Interativo)

Permite a escolha de um rover, uma carga e uma rota. Calcula a distância de ida e volta, valida se o veículo tem capacidade de carga e bateria suficiente para o percurso e analisa os riscos da rota para autorizar ou restringir a missão.

## 3. ESTRUTURAS E KITS DE DADOS UTILIZADOS

Módulos de Residência e Base (residenciakit\_dados.py)

| ID | Nome do Módulo | Capacidade | Ocupação | Energia (kW) | Status | Diagnóstico |
|----|----------------|------------|----------|--------------|--------|-------------|
|----|----------------|------------|----------|--------------|--------|-------------|

|       |               |   |   |       |         |                      |
|-------|---------------|---|---|-------|---------|----------------------|
| HB-01 | Habitat Alpha | 6 | 4 | 18 kW | CRÍTICO | ( $CO_2 > 1000$ ppm) |
|-------|---------------|---|---|-------|---------|----------------------|

|       |             |   |   |       |        |  |
|-------|-------------|---|---|-------|--------|--|
| HB-02 | Laboratório | 4 | 3 | 15 kW | Seguro |  |
|-------|-------------|---|---|-------|--------|--|

|       |            |   |   |       |        |  |
|-------|------------|---|---|-------|--------|--|
| HB-03 | Dormitório | 8 | 6 | 14 kW | Seguro |  |
|-------|------------|---|---|-------|--------|--|

|       |               |   |   |       |        |  |
|-------|---------------|---|---|-------|--------|--|
| HB-04 | Módulo Médico | 3 | 1 | 12 kW | Seguro |  |
|-------|---------------|---|---|-------|--------|--|

|       |                |   |   |      |        |         |
|-------|----------------|---|---|------|--------|---------|
| HB-05 | Módulo Reserva | 5 | 0 | 9 kW | Seguro | (Livre) |
|-------|----------------|---|---|------|--------|---------|

|       |              |   |   |       |        |  |
|-------|--------------|---|---|-------|--------|--|
| HB-06 | Comunicações | 2 | 1 | 11 kW | Seguro |  |
|-------|--------------|---|---|-------|--------|--|

Veículos de Mobilidade (mobilidadekit\_dados.py)

| ID | Modelo | Capacidade (kg) | Autonomia (km) | Bateria (%) |
|----|--------|-----------------|----------------|-------------|
|----|--------|-----------------|----------------|-------------|

RV-01 Scout 180 kg 22.0 km 94%

RV-02 Cargo X1 480 kg 15.0 km 86%

ID Modelo Capacidade (kg) Autonomia (km) Bateria (%)

RV-03 Cargo X2 650 kg 11.5 km 78%

RV-04 Explorer 120 kg 28.0 km 91%

RV-05 Crew Rover 300 kg 18.5 km 83%

RV-06 Utility 360 kg 16.0 km 89%

#### 4. CÓDIGO-FONTE DO PROGRAMA 1 (KIT DE DADOS — MOBILIDADE)

# Bases para projetos de transporte, exploração, seleção de veículos e análise de rotas.

```
veiculos = [
```

```
    {"id": "RV-01", "modelo": "Scout", "capacidade_kg": 180, "autonomia_km": 22.0, "bateria_pct": 94,
"velocidade_kmh": 8},
```

```
    {"id": "RV-02", "modelo": "Cargo X1", "capacidade_kg": 480, "autonomia_km": 15.0, "bateria_pct": 86,
"velocidade_kmh": 6},
```

```
    {"id": "RV-03", "modelo": "Cargo X2", "capacidade_kg": 650, "autonomia_km": 11.5, "bateria_pct": 78,
"velocidade_kmh": 5},
```

```
    {"id": "RV-04", "modelo": "Explorer", "capacidade_kg": 120, "autonomia_km": 28.0, "bateria_pct": 91,
"velocidade_kmh": 10},
```

```
    {"id": "RV-05", "modelo": "Crew Rover", "capacidade_kg": 300, "autonomia_km": 18.5, "bateria_pct": 83,
"velocidade_kmh": 7},
```

```
    {"id": "RV-06", "modelo": "Utility", "capacidade_kg": 360, "autonomia_km": 16.0, "bateria_pct": 89,
"velocidade_kmh": 6}
```

```
]
```

```
rotas = [
```

```
    {"id": "RT-01", "origem": "Base Alpha", "destino": "Mina A", "distancia_km": 7.2, "inclinacao_max_graus": 8,
"risco": "baixo"},
```

```
    {"id": "RT-02", "origem": "Base Alpha", "destino": "Mina B", "distancia_km": 10.8, "inclinacao_max_graus": 14,
"risco": "medio"},
```

```
    {"id": "RT-03", "origem": "Mina A", "destino": "Deposito", "distancia_km": 5.4, "inclinacao_max_graus": 10,
"risco": "baixo"},
```

```
    {"id": "RT-04", "origem": "Deposito", "destino": "Base Alpha", "distancia_km": 6.1, "inclinacao_max_graus": 17,
"risco": "alto"},
```

```
{ "id": "RT-05", "origem": "Base Alpha", "destino": "Cratera Sul", "distancia_km": 13.5, "inclinacao_max_graus": 12, "risco": "medio"},
```

```
{ "id": "RT-06", "origem": "Mina B", "destino": "Deposito", "distancia_km": 8.6, "inclinacao_max_graus": 19, "risco": "alto"}
```

```
]
```

```
cargas = [
```

```
{ "id": "CG-01", "tipo": "regolito", "massa_kg": 120, "prioridade": 2},
```

```
{ "id": "CG-02", "tipo": "gelo extraido", "massa_kg": 260, "prioridade": 1},
```

```
{ "id": "CG-03", "tipo": "baterias", "massa_kg": 180, "prioridade": 1},
```

```
{ "id": "CG-04", "tipo": "pecas mecanicas", "massa_kg": 310, "prioridade": 2},
```

```
{ "id": "CG-05", "tipo": "alimentos", "massa_kg": 95, "prioridade": 1},
```

```
{ "id": "CG-06", "tipo": "equipamentos cientificos", "massa_kg": 220, "prioridade": 3}
```

```
]
```

```
pontos_interesse = [
```

```
{ "nome": "Base Alpha", "coordenada": (10, 12)},
```

```
{ "nome": "Mina A", "coordenada": (18, 21)},
```

```
{ "nome": "Mina B", "coordenada": (25, 28)},
```

```
{ "nome": "Deposito", "coordenada": (15, 18)},
```

```
{ "nome": "Cratera Sul", "coordenada": (31, 9)},
```

```
{ "nome": "Estacao Energia", "coordenada": (8, 16)}
```

```
]
```

```
leituras_operacionais = [
```

```
{ "ciclo": 1, "veiculo": "RV-02", "bateria_pct": 86, "temp_motor_c": 42, "consumo_pct_km": 4.1},
```

```
{ "ciclo": 2, "veiculo": "RV-02", "bateria_pct": 78, "temp_motor_c": 45, "consumo_pct_km": 4.4},
```

```
{ "ciclo": 3, "veiculo": "RV-02", "bateria_pct": 69, "temp_motor_c": 47, "consumo_pct_km": 4.6},
```

```
{ "ciclo": 4, "veiculo": "RV-02", "bateria_pct": 61, "temp_motor_c": 51, "consumo_pct_km": 4.9},
```

```
    {"ciclo": 5, "veiculo": "RV-02", "bateria_pct": 54, "temp_motor_c": 54, "consumo_pct_km": 5.1},  
    {"ciclo": 6, "veiculo": "RV-02", "bateria_pct": 46, "temp_motor_c": 56, "consumo_pct_km": 5.3}  
]
```

# 0 = perigoso | 1 = seguro | 2 = atencao

```
mapa_terreno = [  
    [1, 1, 1, 2, 2, 0],  
    [1, 1, 2, 2, 1, 0],  
    [1, 2, 2, 1, 1, 1],  
    [1, 1, 1, 1, 2, 2],  
    [2, 2, 1, 1, 1, 0],  
    [0, 2, 2, 1, 1, 1]  
]
```

# Referencia historica NASA/LROC: distancias percorridas por rovers lunares.

```
historico_rovers_lunares = [  
    {"veiculo": "Lunokhod 1", "ano": 1970, "distancia_km": 10.54},  
    {"veiculo": "Apollo 15 LRV", "ano": 1971, "distancia_km": 27.80},  
    {"veiculo": "Apollo 16 LRV", "ano": 1972, "distancia_km": 26.70},  
    {"veiculo": "Apollo 17 LRV", "ano": 1972, "distancia_km": 35.74},  
    {"veiculo": "Lunokhod 2", "ano": 1973, "distancia_km": 42.15}  
  
from mobilidadekit_dados import veiculos, leituras_operacionais, mapa_terreno  
from residenciakit_dados import modulos, leituras_ambientais, mapa_base  
  
limites_seguranca = (1000.0, 20.5, 80.0)  
  
ARTE_FINAL = r"""
```



```
def classificar_rover(bateria_pct):

    bateria_minima = limites_seguranca[2]

    if bateria_pct >= bateria_minima:

        return "Rover seguro para operação"

    elif bateria_pct >= 50.0:

        return "ATENÇÃO. Rover em condição de ALERTA"

    else:

        return "ALERTA!!! Rover em condição CRÍTICA!"
```

```
def classificar_modulo(modulo_id):

    co2_limite = limites_seguranca[0]

    o2_limite = limites_seguranca[1]

    leituras_modulo = [leitura for leitura in leituras_ambientais if leitura["modulo"] == modulo_id]

    if not leituras_modulo:
```

```
return "Nenhuma leitura disponível para o módulo especificado."
```

```
ultima_leitura = leituras_modulo[-1]
```

```
if ultima_leitura["co2_ppm"] > co2_limite or ultima_leitura["o2_pct"] < o2_limite:
```

```
    return f"ALERTA!!! (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%) - Condição CRÍTICA!"
```

```
else:
```

```
    return f"Módulo seguro para operação (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%)"
```

```
def calcular_indicadores_globais():
```

```
    total_capacidade = sum(modulo["capacidade"] for modulo in modulos)
```

```
    total_ocupacao = sum(modulo["ocupacao"] for modulo in modulos)
```

```
    taxa_ocupacao = (total_ocupacao / total_capacidade) * 100 if total_capacidade > 0 else 0
```

```
    total_autonomia = sum(veiculo["autonomia_km"] for veiculo in veiculos)
```

```
    media_autonomia = total_autonomia / len(veiculos) if veiculos else 0
```

```
    vagas_totais = sum(modulo["capacidade"] - modulo["ocupacao"] for modulo in modulos)
```

```
    return taxa_ocupacao, media_autonomia, vagas_totais
```

```
def analisar_matriz_terreno(matriz):
```

```
    seguro = 0
```

```
    atencao = 0
```

```
    perigoso = 0
```

```
for linha in matriz:
    for celula in linha:
        if celula == 1:
            seguro += 1
        elif celula == 2:
            atencao += 1
        elif celula == 0:
            perigoso += 1
    return seguro, atencao, perigoso
```

```
def executar_diagnostico():
    print("=====")
    print("    CENTRO DE CONTROLE LUNAR | DIAGNÓSTICO OPERACIONAL    ")
    print("=====")

    print("Status dos módulos de residência e habitat")
    modulos_alerta = 0
    for modulo in modulos:
        vagas = modulo["capacidade"] - modulo["ocupacao"]
        status_ambiental = classificar_modulo(modulo["id"])

        if "CRÍTICA" in status_ambiental:
            modulos_alerta += 1
            print(f"ALERTA!!! Módulo {modulo['nome']} em condição CRÍTICA! (Vagas: {vagas})")
    print("\n" + "-" * 66 + "\n")
```

```
print("Status dos rovers e veículos de transporte")
```

```
rovers_aptos = 0
```

```
for veiculo in veiculos:
```

```
    status_rover = classificar_rover(veiculo["bateria_pct"])
```

```
    if "seguro" in status_rover:
```

```
        rovers_aptos += 1
```

```
    print(f'Rover {veiculo["modelo"]} (Bateria: {veiculo["bateria_pct"]}%) - {status_rover}')
```

```
print("\n" + "-" * 66 + "\n")
```

```
seg, atenc, per = analisar_matriz_terreno(mapa_terreno)
```

```
print("Mapeamento do terreno ao redor da base")
```

```
print(f'Seguros: {seg}, Atenção: {atenc}, Perigosos: {per}')
```

```
print("\n" + "-" * 66 + "\n")
```

```
taxa_ocupacao, media_autonomia, vagas_totais = calcular_indicadores_globais()
```

```
print("Indicadores globais da base lunar")
```

```
print(f'Taxa de ocupação da base: {taxa_ocupacao:.1f}%')
```

```
print(f'Média de autonomia dos rovers: {media_autonomia:.2f} km')
```

```
print(f'Vagas totais disponíveis nos módulos: {vagas_totais}')
```

```
print("\n" + "-" * 66 + "\n")
```

```
print("Diagnóstico completo. Encerrando o relatório do Centro de Controle Lunar.")
```

```
if modulos_alerta > 0:
```

```
    print(f'ALERTA!!! Existem {modulos_alerta} módulos em condição CRÍTICA. Ação imediata recomendada.')

```

```
    print(f'Existem {vagas_totais} vagas disponíveis e {rovers_aptos} rovers APTOS para realocação/resgate.')

```

```
    print("Recomendação: Execute o 'Programa 02' para planejar a evacuação dos módulos afetados.")
```

```
else:
```

```
print("Todos os módulos estão operando dentro dos parâmetros de segurança. Nenhuma ação corretiva necessária.")
```

```
print(ARTE_FINAL)
```

```
if __name__ == "__main__":
```

```
    executar_diagnostico()
```

```
]
```

## 2. CÓDIGOS-FONTE: KIT DE DADOS — RESIDENCIA E BASE

# Bases para projetos de habitat, capacidade, suporte a vida e distribuicao de tripulacao.

```
modulos = [
```

```
    {"id": "HB-01", "nome": "Habitat Alpha", "capacidade": 6, "ocupacao": 4, "energia_kw": 18},
```

```
    {"id": "HB-02", "nome": "Laboratorio", "capacidade": 4, "ocupacao": 3, "energia_kw": 15},
```

```
    {"id": "HB-03", "nome": "Dormitorio", "capacidade": 8, "ocupacao": 6, "energia_kw": 14},
```

```
    {"id": "HB-04", "nome": "Modulo Medico", "capacidade": 3, "ocupacao": 1, "energia_kw": 12},
```

```
    {"id": "HB-05", "nome": "Modulo Reserva", "capacidade": 5, "ocupacao": 0, "energia_kw": 9},
```

```
    {"id": "HB-06", "nome": "Comunicacoes", "capacidade": 2, "ocupacao": 1, "energia_kw": 11}
```

```
]
```

```
leituras_ambientais = [
```

```
    {"ciclo": 1, "modulo": "HB-01", "o2_pct": 20.8, "co2_ppm": 690, "temp_c": 22.4, "pressao_kpa": 101.1, "umidade_pct": 42},
```

```
    {"ciclo": 2, "modulo": "HB-01", "o2_pct": 20.7, "co2_ppm": 720, "temp_c": 22.8, "pressao_kpa": 101.0, "umidade_pct": 43},
```

```
    {"ciclo": 3, "modulo": "HB-01", "o2_pct": 20.6, "co2_ppm": 810, "temp_c": 23.0, "pressao_kpa": 100.9, "umidade_pct": 44},
```

```
    {"ciclo": 4, "modulo": "HB-01", "o2_pct": 20.5, "co2_ppm": 890, "temp_c": 23.4, "pressao_kpa": 100.7, "umidade_pct": 45},
```

```
    {"ciclo": 5, "modulo": "HB-01", "o2_pct": 20.4, "co2_ppm": 960, "temp_c": 23.8, "pressao_kpa": 100.5, "umidade_pct": 46},
```

```
    {"ciclo": 6, "modulo": "HB-01", "o2_pct": 20.3, "co2_ppm": 1080, "temp_c": 24.1, "pressao_kpa": 100.2, "umidade_pct": 47}
```

```
]
```

```
tripulacao = [
```

```
    {"id": "TR-01", "funcao": "Comandante", "modulo": "HB-01", "turno": "A"},
```

```

{"id": "TR-02", "funcao": "Engenheira", "modulo": "HB-01", "turno": "A"},
{"id": "TR-03", "funcao": "Geologo", "modulo": "HB-02", "turno": "B"},
{"id": "TR-04", "funcao": "Medica", "modulo": "HB-04", "turno": "A"},
{"id": "TR-05", "funcao": "Tecnico", "modulo": "HB-03", "turno": "B"},
{"id": "TR-06", "funcao": "Pesquisadora", "modulo": "HB-02", "turno": "B"}
]

```

```

consumo_modulos = [
    {"modulo": "HB-01", "agua_l_dia": 52, "energia_kwh_dia": 410},
    {"modulo": "HB-02", "agua_l_dia": 31, "energia_kwh_dia": 360},
    {"modulo": "HB-03", "agua_l_dia": 68, "energia_kwh_dia": 330},
    {"modulo": "HB-04", "agua_l_dia": 18, "energia_kwh_dia": 290},
    {"modulo": "HB-05", "agua_l_dia": 7, "energia_kwh_dia": 145},
    {"modulo": "HB-06", "agua_l_dia": 5, "energia_kwh_dia": 180}
]

```

# 0 = livre | 1 = ocupado | 2 = acesso restrito

```

mapa_base = [
    [1, 1, 0, 0, 2, 2],
    [1, 1, 1, 0, 0, 2],
    [0, 1, 1, 1, 0, 0],
    [0, 0, 1, 1, 1, 0],
    [2, 0, 0, 1, 1, 1],
    [2, 2, 0, 0, 1, 1]
]

```

# Referencias NASA para o ambiente lunar.

```

ambiente_lunar = {
    "gravidade_relativa_terra": 1 / 6,
    "temperatura_sol_c": 127,
    "temperatura_escuridao_c": -173
}

```

# Referencia historica NASA: permanencia e atividade extraveicular nas missoes Apollo.

```

historico_apollo_superficie = [
    {"missao": "Apollo 11", "horas_superficie": 21.5, "numero_evas": 1, "horas_eva": 2.5},
    {"missao": "Apollo 12", "horas_superficie": 31.5, "numero_evas": 2, "horas_eva": 7.8},
    {"missao": "Apollo 14", "horas_superficie": 33.5, "numero_evas": 2, "horas_eva": 9.4},
    {"missao": "Apollo 15", "horas_superficie": 67.0, "numero_evas": 3, "horas_eva": 19.1},
    {"missao": "Apollo 16", "horas_superficie": 71.0, "numero_evas": 3, "horas_eva": 20.2},
    {"missao": "Apollo 17", "horas_superficie": 75.0, "numero_evas": 3, "horas_eva": 22.1}
]

```

```
from mobilidadekit_dados import veiculos, leituras_operacionais, mapa_terreno
from residenciakit_dados import modulos, leituras_ambientais, mapa_base
```

```
limites_seguranca = (1000.0, 20.5, 80.0)
```

```
def classificar_rover(bateria_pct):
    bateria_minima = limites_seguranca[2]
    if bateria_pct >= bateria_minima:
        return "Rover seguro para operação"
    elif bateria_pct >= 50.0:
        return "ATENÇÃO. Rover em condição de ALERTA"
    else:
        return "ALERTA!!! Rover em condição CRÍTICA!"
```

```
def classificar_modulo(modulo_id):
    co2_limite = limites_seguranca[0]
    o2_limite = limites_seguranca[1]
```

```
    leituras_modulo = [leitura for leitura in leituras_ambientais if leitura["modulo"] == modulo_id]
    if not leituras_modulo:
        return "Nenhuma leitura disponível para o módulo especificado."
```

```
    ultima_leitura = leituras_modulo[-1]
```

```
    if ultima_leitura["co2_ppm"] > co2_limite or ultima_leitura["o2_pct"] < o2_limite:
        return f"ALERTA!!! (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%) - Condição CRÍTICA!"
    else:
        return f"Módulo seguro para operação (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%)"
```

```
def calcular_indicadores_globais():
    total_capacidade = sum(modulo["capacidade"] for modulo in modulos)
    total_ocupacao = sum(modulo["ocupacao"] for modulo in modulos)
    taxa_ocupacao = (total_ocupacao / total_capacidade) * 100 if total_capacidade > 0 else 0

    total_autonomia = sum(veiculo["autonomia_km"] for veiculo in veiculos)
    media_autonomia = total_autonomia / len(veiculos) if veiculos else 0
```

```
vagas_totais = sum(modulo["capacidade"] - modulo["ocupacao"] for modulo in modulos)
```

```
return taxa_ocupacao, media_autonomia, vagas_totais
```

```
def analisar_matriz_terreno(matriz):
```

```
    seguro = 0
```

```
    atencao = 0
```

```
    perigoso = 0
```

```
    for linha in matriz:
```

```
        for celula in linha:
```

```
            if celula == 1:
```

```
                seguro += 1
```

```
            elif celula == 2:
```

```
                atencao += 1
```

```
            elif celula == 0:
```

```
                perigoso += 1
```

```
    return seguro, atencao, perigoso
```

```
def executar_diagnostico():
```

```
    print("=====")
```

```
    print("    CENTRO DE CONTROLE LUNAR | DIAGNÓSTICO OPERACIONAL    ")
```

```
    print("=====")
```

```
    print("Status dos módulos de residência e habitat")
```

```
    modulos_alerta = 0
```

```
    for modulo in modulos:
```

```
        vagas = modulo["capacidade"] - modulo["ocupacao"]
```

```
        status_ambiental = classificar_modulo(modulo["id"])
```

```
        if "CRÍTICA" in status_ambiental:
```

```
            modulos_alerta += 1
```

```
            print(f"ALERTA!!! Módulo {modulo['nome']} em condição CRÍTICA! (Vagas: {vagas})")
```

```
    print("\n" + "-" * 66 + "\n")
```

```
    print("Status dos rovers e veículos de transporte")
```

```
    rovers_aptos = 0
```

```
    for veiculo in veiculos:
```

```
        status_rover = classificar_rover(veiculo["bateria_pct"])
```

```
        if "seguro" in status_rover:
```

```

    rovers_aptos += 1
    print(f"Rover {veiculo['modelo']} (Bateria: {veiculo['bateria_pct']}%) - {status_rover}")
print("\n" + "-" * 66 + "\n")

seg, atenc, per = analisar_matriz_terreno(mapa_terreno)
print("Mapeamento do terreno ao redor da base")
print(f"Seguros: {seg}, Atenção: {atenc}, Perigosos: {per}")
print("\n" + "-" * 66 + "\n")

taxa_ocupacao, media_autonomia, vagas_totais = calcular_indicadores_globais()
print("Indicadores globais da base lunar")
print(f"Taxa de ocupação da base: {taxa_ocupacao:.1f}%")
print(f"Média de autonomia dos rovers: {media_autonomia:.2f} km")
print(f"Vagas totais disponíveis nos módulos: {vagas_totais}")
print("\n" + "-" * 66 + "\n")

print("Diagnóstico completo. Encerrando o relatório do Centro de Controle Lunar.")
if modulos_alerta > 0:
    print(f"ALERTA!!! Existem {modulos_alerta} módulos em condição CRÍTICA. Ação imediata recomendada.")
    print(f"Existem {vagas_totais} vagas disponíveis e {rovers_aptos} rovers APTOS para realocação/resgate.")
    print("Recomendação: Execute o 'Programa 02' para planejar a evacuação dos módulos afetados.")
else:
    print("Todos os módulos estão operando dentro dos parâmetros de segurança. Nenhuma ação corretiva necessária.")

```